Graph Algorithms

Part-2: Shortest paths and Minimum Spanning Tree

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline

1. **Shortest Path Algorithms**
   - Dijkstra
   - Bellman–Ford
   - Floyd–Warshall

2. **Minimum spanning tree**
   - Kruskal
   - Prim

ltsach@hcmut.edu.vn
Thanh-Sach LE

Shortest Path Algorithms

ltsach@hcmut.edu.vn
Thanh-Sach LE

Shortest Path Algorithms

Need vs algorithm

Need (what you want)	**Algorithm**
Unweighted graph (or all edges equal cost): shortest path = minimum number of edges	BFS *(Graph traversal)*
Single-source; nonnegative edge weights	Dijkstra
Single-source; weights may be negative; detect negative cycles	Bellman–Ford
All pairs (u, v); typically small/dense n	Floyd–Warshall

- **BFS** was covered with DFS/BFS traversal; the three algorithms below extend shortest paths to weighted settings.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra

Introduction

- Given weighted graph with nonnegative weights.
- Compute shortest distance from source s to every vertex.
- Dijkstra fails with negative edge weights.

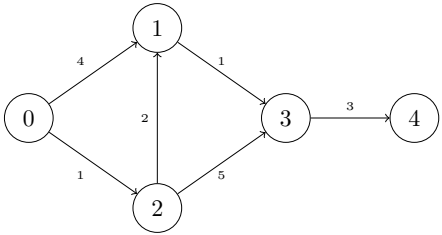
ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra

Example

Problem.

- Directed graph with nonnegative edge weights (figure).
- **Source:** $s = 0$.
- Compute shortest distance from s to each vertex, and *one* concrete shortest path.



Result from source 0.

v	dist	shortest path
0	0	0
1	3	$0 \to 2 \to 1$
2	1	$0 \to 2$
3	4	$0 \to 2 \to 1 \to 3$
4	7	$0 \to 2 \to 1 \to 3 \to 4$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra

Core Idea

- Maintain tentative distances `dist[v]`.
- Repeatedly pick unsettled vertex with smallest distance.
- Relax outgoing edges:

$$\text{if } dist[u] + w < dist[v] \text{ then update } dist[v]$$

Meaning of the Relaxation Formula

`dist[u] + w` is the cost of going from source to `u`, then taking edge `(u,v)` with weight `w`. If this new candidate is smaller than current `dist[v]`, we found a better route to `v`, so we update `dist[v]` (and set `parent[v] = u` for path reconstruction).

Shortest paths and MST

Outline

Shortest Path Algorithms

6 Dijkstra

Bellman–Ford

Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra

Relaxation Rule

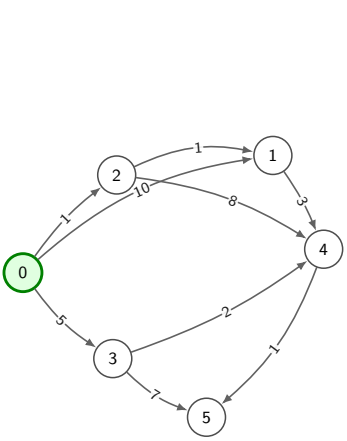
```
if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
    parent[v] = u;
}
```

- This local rule drives the entire algorithm.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough: graph and idea



L0

L1

L2

L3

L4

priority queue

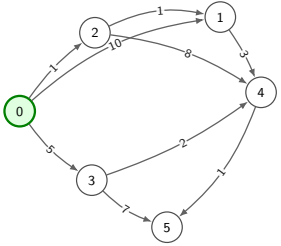
- **Problem:** from source 0, find shortest paths to all other vertices.
- **Colors:** blue = current node; green = relax targets / in queue; orange = settled.
- **Queue** pq**:** stores pairs `(path-cost, vertex)` and always pops the smallest-cost candidate first.

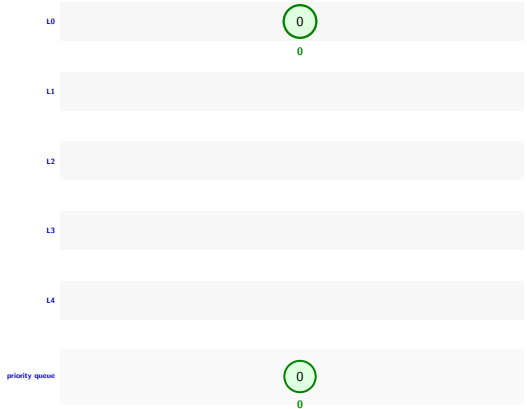
ltsach@hcmut.edu.vn

Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (1/9)

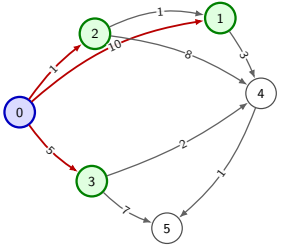


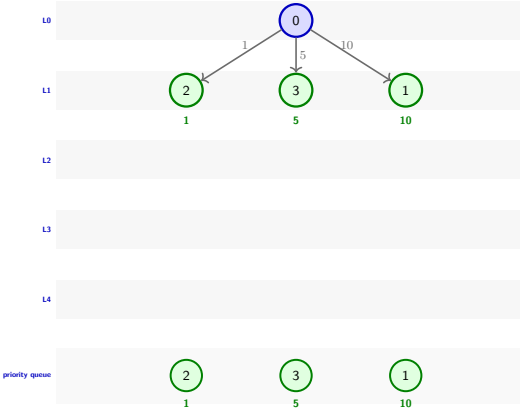


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (2/9)

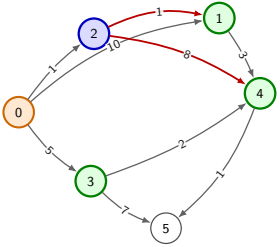


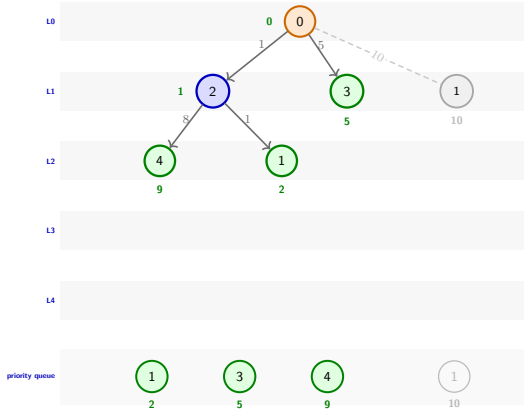


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (3/9)

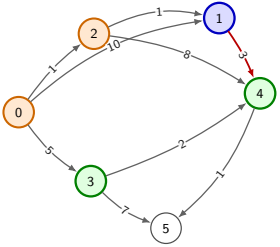


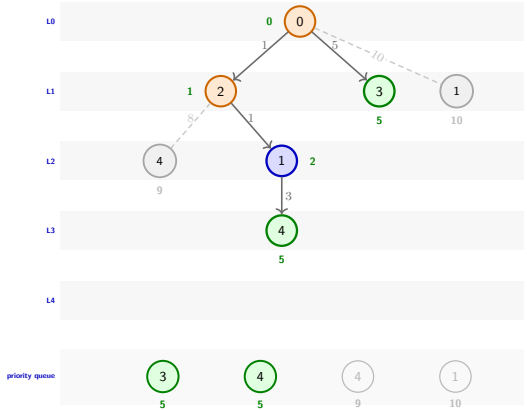


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (4/9)

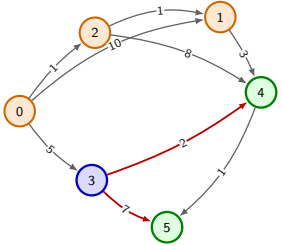


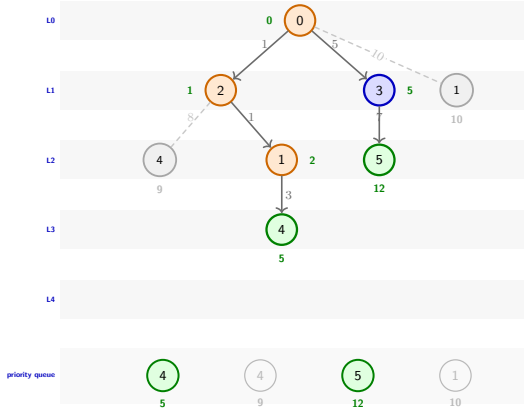


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (5/9)

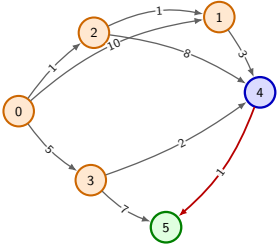


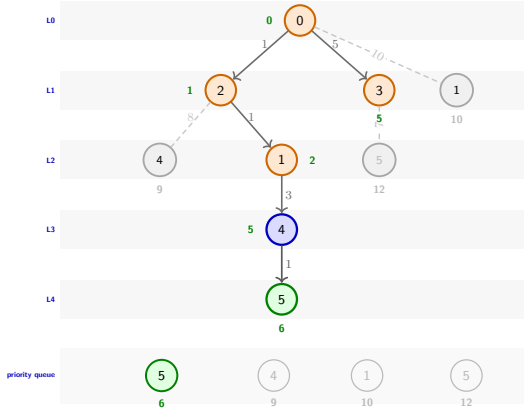


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (6/9)



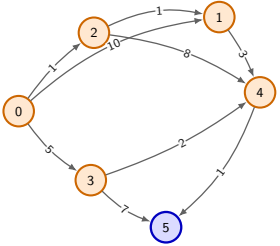


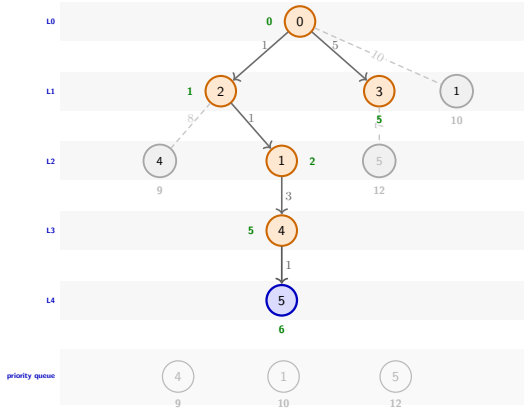
ltsach@hcmut.edu.vn

Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (7/9)

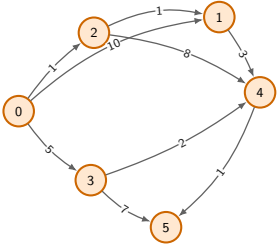


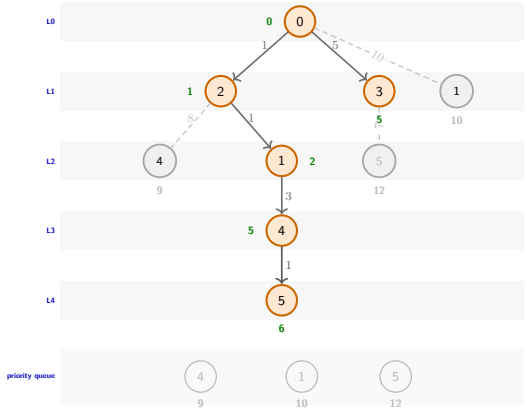


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (8/9)

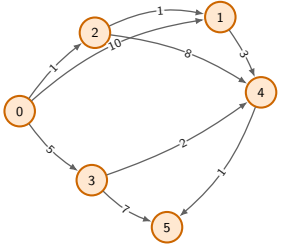


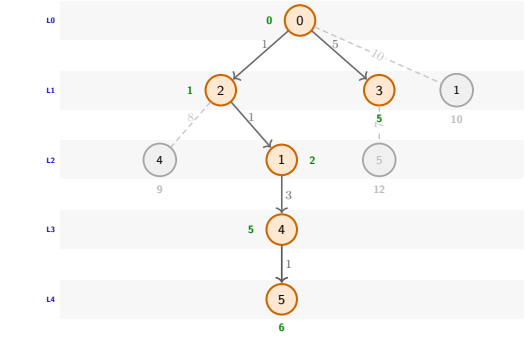


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Walkthrough (9/9)





We may keep popping leftover queue entries; if that vertex was already finalized, relaxation does nothing.

ltsach@hcmut.edu.vn

Thanh-Sach LE

Dijkstra with Min-Priority Queue

Function signature

```
vector<int> dijkstra(int s, const vector<vector<pair<int,int>>>&
    adj);
```

- **Return value:** `dist[i]` = weight of a shortest path from s to i (or `INF` if i is unreachable).
- `s`: index of the source vertex; must satisfy $0 \leq s <$ `adj.size()`.
- `adj`: weighted adjacency list passed by `const` reference (no copy); layout on the next slide.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Parameter `adj`

- `adj.size()` is n (number of vertices, labeled $0..n-1$).
- `adj[u]` lists outgoing edges from u: each element is `pair<int,int>` = `{v, w}` (neighbor v, weight $w \geq 0$).

```
// Example: 3 vertices; edges 0->1 (4), 0->2 (1), 2->1 (2)
vector<vector<pair<int,int>>> adj = {
    { {1, 4}, {2, 1} }, // from 0
    { }, // from 1 (none)
    { {1, 2} } // from 2
};
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Local state (1)

```
int n = adj.size();
const int INF = 1e9;
vector<int> dist(n, INF), parent(n, -1);
```

- `n`: vertex count; bounds array sizes and loops.
- `INF`: "infinity" sentinel; must exceed any real shortest-path weight used in comparisons (here `1e9`).
- `dist[i]`: current best distance $s \rightsquigarrow i$; starts at `INF` except `dist[s]` later set to 0.
- `parent[i]`: predecessor of i on a cheapest path (for path reconstruction); `-1` means unknown or source.

Shortest paths and MST

Outline

Shortest Path Algorithms

20 Dijkstra

Bellman–Ford

Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn

Thanh-Sach LE

Dijkstra with Min-Priority Queue

Local state (2a): `priority_queue` template

```
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> pq;
```

- `pair<int,int>` **(1st template argument):** type of each stored element. We use {`.first, .second`} = {`tentative distance, vertex id`} and push {`dist[v], v`}.
- `vector<pair<int,int>>` **(2nd template argument):** internal container backing the binary heap (array-based). The queue is not a sorted `vector`, but a heap order in that array. `vector` is the standard choice (also the default if you omit this parameter).
- `greater<pair<int,int>>` **(3rd template argument):** comparison functor. It makes `top()` return the **smallest** key — a **min**-heap. For `pair`, `greater` uses lexicographic order: smaller `.first` has higher priority; ties broken by `.second`.

Shortest paths and MST

Outline

Shortest Path Algorithms

21 Dijkstra

Bellman–Ford

Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn

Thanh-Sach LE

Dijkstra with Min-Priority Queue

Local state (2b): min-heap example

- Smaller `.first` (distance) is extracted first, which matches Dijkstra's "always settle the closest unsettled vertex."
- Older duplicate pairs for the same vertex may remain in the heap; code skips stale pops with `du != dist[u]`.

```
// Two candidates: top() is {3,1} (smaller first component)
pq.push({3, 1});
pq.push({7, 4});
// pq.top() == {3,1}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Main procedure (1)

- Initialize unreachable distances: every `dist[i] = INF`, then `dist[s] = 0`.
- Push the seed `{0, s}` to `pq`: the only known path is at the source with cost 0.
- **Meaning of the heap:** it holds *candidates* "we have found a path of total weight `d` to `u`"; Dijkstra always extracts the smallest `d` next (nonnegative weights make that distance final).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Main procedure (2)

- **Pop** {`du, u`}. If `du != dist[u]`, discard (**lazy deletion**: an older, worse entry for u).
- Otherwise relax each outgoing {`v, w`} from `adj[u]`: if `dist[u] + w < dist[v]`, set `dist[v]`, `parent[v] = u`, and `pq.push({dist[v], v})`.
- Repeat until `pq` is empty. Invariant (nonnegative w): when an entry is not stale, `dist[u]` equals the true shortest distance from s.
- *Relaxation example:* `dist[u] = 5`, edge to v with `w = 2` $\Rightarrow$ candidate 7 for v; update + push only if 7 improves `dist[v]`.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Code

```
vector<int> dijkstra(int s, const vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    const int INF = 1e9;
    vector<int> dist(n, INF), parent(n, -1);

    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra with Min-Priority Queue

Code (continued)

```
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != dist[u]) continue; // lazy deletion

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra Complexity

- Naive selection: $O(n^2 + m)$.
- Priority queue version: $O((n + m) \log n)$.
- Often written as $O(m \log n)$ for sparse connected graphs.
- Space: $O(n + m)$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Dijkstra Representation Notes

- Weighted adjacency list is the natural structure.
- Matrix + naive selection gives $O(n^2)$ and may be fine for dense graphs.
- Priority queue gains are strongest on sparse graphs.
- BFS is valid only when all weights are equal.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Negative Weight Warning

Requirement

Dijkstra requires all edge weights to be nonnegative.

- With negative edges, use Bellman-Ford.
- For other settings, choose algorithm by constraints.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Common Dijkstra Mistakes

- Using Dijkstra with negative edges.
- Not initializing INF correctly.
- Overflow in `dist[u] + w` with small integer type.
- Not skipping outdated priority queue entries.
- Assuming BFS solves weighted shortest path.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Definition

- Single-source shortest paths allowing negative edge weights.
- Also detects negative cycles reachable from source.

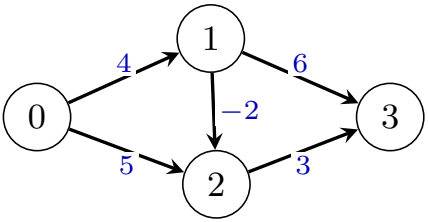
ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Example

Problem.

- Directed graph; weights may be negative (figure). No negative cycle in this instance.
- **Source:** $s = 0$.
- Compute shortest distance from s to each vertex, and *one* concrete shortest path.



Result from source 0.

v	dist	shortest path
0	0	0
1	4	$0 \to 1$
2	2	$0 \to 1 \to 2$
3	5	$0 \to 1 \to 2 \to 3$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Core idea and basic steps

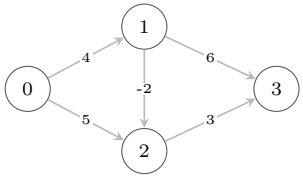
1. Initialize distances: $dist[s] = 0$, others $= \infty$.
2. Repeat relaxation over all edges exactly $n - 1$ passes.
3. Extra pass: if any edge still relaxes $\Rightarrow$ negative cycle exists.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 1/8 — initialize distances

Bellman–Ford example graph



State after current relax step

Pass: init Current edge: -

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	INF	INF	INF

Edge scan order (per pass; by source u):

1. $0 \to 1(4)$
2. $0 \to 2(5)$
3. $1 \to 2(-2)$
4. $1 \to 3(6)$
5. $2 \to 3(3)$

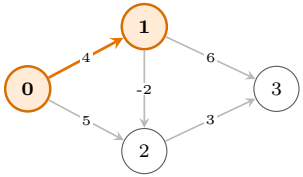
Initialize: dist[s]=0, others=INF.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 2/8 — relax edge 0 → 1

Bellman–Ford example graph



State after current relax step

Pass: 1 Current edge: 0 → 1 (4)

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	INF	INF

Edge scan order (per pass; by source u):

▷ 1. 0 → 1(4)
2. 0 → 2(5)
3. 1 → 2(-2)
4. 1 → 3(6)
5. 2 → 3(3)

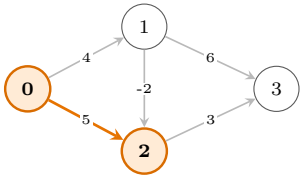
Relax 0->1: update dist[1] = 4.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 3/8 — relax edge 0 → 2

Bellman–Ford example graph



State after current relax step

Pass: 1 Current edge: 0 → 2 (5)

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	5	INF

Edge scan order (per pass; by source u):

1. 0 → 1(4)
2. ▷ **2. 0 → 2(5)**
3. 1 → 2(-2)
4. 1 → 3(6)
5. 2 → 3(3)

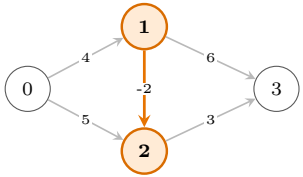
Relax 0->2: update dist[2] = 5.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 4/8 — relax edge 1 → 2

Bellman–Ford example graph



State after current relax step

Pass: 1 Current edge: 1 → 2 (-2)

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	2	INF

Edge scan order (per pass; by source u):

1. 0 → 1(4)
2. 0 → 2(5)
3. ▷ **1 → 2(-2)**
4. 1 → 3(6)
5. 2 → 3(3)

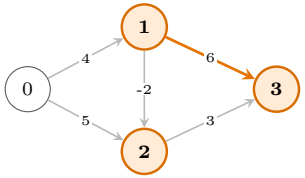
Relax 1->2: improve dist[2] from 5 to 2.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 5/8 — relax edge 1 → 3

Bellman–Ford example graph



State after current relax step

Pass: 1 Current edge: 1 → 3 (6)

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	2	10

Edge scan order (per pass; by source u):

1. 0 → 1(4)
2. 0 → 2(5)
3. 1 → 2(-2)
4. ▷ 1 → 3(6)
5. 2 → 3(3)

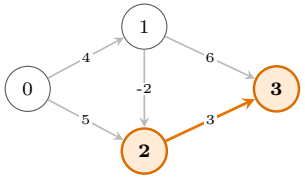
Relax 1->3: update dist[3] from INF to 4+6 = 10 (later tightened).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 6/8 — relax edge 2 → 3

Bellman–Ford example graph



State after current relax step

Pass: 1 Current edge: 2 → 3 (3)

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	2	5

Edge scan order (per pass; by source u):

1. 0 → 1(4)
2. 0 → 2(5)
3. 1 → 2(-2)
4. 1 → 3(6)
5. ▷ **5.** 2 → 3**(3)**

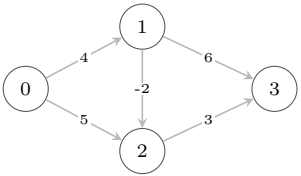
Relax 2->3: improve dist[3] from 10 to 2+3 = 5.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 7/8 — second pass (no update)

Bellman–Ford example graph



State after current relax step

Pass: 2 Current edge: all edges

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	2	5

Edge scan order (per pass; by source u):

1. $0 \to 1(4)$
2. $0 \to 2(5)$
3. $1 \to 2(-2)$
4. $1 \to 3(6)$
5. $2 \to 3(3)$

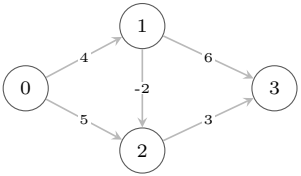
Pass 2: no update => distances stabilized.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Step 8/8 — extra check pass

Bellman–Ford example graph



State after current relax step

Pass: check Current edge: all edges

$d[0]$	$d[1]$	$d[2]$	$d[3]$
0	4	2	5

Edge scan order (per pass; by source u):

1. $0 \to 1(4)$
2. $0 \to 2(5)$
3. $1 \to 2(-2)$
4. $1 \to 3(6)$
5. $2 \to 3(3)$

Extra check pass: still no update => no reachable negative cycle.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Function signature and return type

```
struct Edge { int u, v, w; };
struct BFResult { vector<int> dist, parent; bool negCycle; };
BFResult bellmanFord(
    int n, int s, const vector<Edge>& edges);
```

- `n`: number of vertices `0..n-1`.
- `s`: source vertex.
- `edges`: directed weighted edge list.
- Return:
  - `dist[v]`: shortest distance estimate from `s` to `v`
  - `parent[v]`: predecessor to reconstruct path
  - `negCycle`: reachable negative cycle detected or not

Shortest paths and MST

Outline

Shortest Path Algorithms

Dijkstra

42 Bellman–Ford

Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Important internal variables (with example)

```
const int INF = 1e9;
vector<int> dist(n, INF); // shortest-known distance from s
vector<int> parent(n, -1); // predecessor for path reconstruction
dist[s] = 0;
bool any = false; // did this pass improve something?
bool negCycle = false; // true if extra pass still relaxes
```

- Example (n=4, s=0): initially `dist = [0, INF, INF, INF]`.
- After some relaxations, it may become `dist = [0,4,2,5]` and `parent = [-1,0,1,2]`.
- If one extra pass still decreases any entry, set `negCycle = true`.

Shortest paths and MST

Outline

Shortest Path Algorithms

Dijkstra

43 Bellman–Ford

Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Implementation (part 1)

```
struct Edge { int u, v, w; };
struct BFResult { vector<int> dist, parent; bool negCycle; };
BFResult bellmanFord(int n, int s, const vector<Edge>& edges) {
    const int INF = 1e9;
    vector<int> dist(n, INF), parent(n, -1);
    dist[s] = 0;
    for (int it = 1; it <= n - 1; ++it) {
        bool any = false;
        for (auto [u,v,w] : edges) if (dist[u] < INF && dist[u] + w <
             dist[v]) {
            dist[v] = dist[u] + w;
            parent[v] = u;
            any = true;
        }
        if (!any) break;
    }
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Implementation (part 2)

```
// continuation after part 1
    bool negCycle = false;
    for (auto [u,v,w] : edges) {
        if (dist[u] < INF && dist[u] + w < dist[v]) {
            negCycle = true;
            break;
        }
    }
    return {dist, parent, negCycle};
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Path reconstruction from parent

```
vector<int> buildPath(int s, int t, const vector<int>& parent, const
    vector<int>& dist) {
    const int INF = 1e9;
    if (dist[t] >= INF) return {}; // unreachable
    vector<int> path;
    for (int cur = t; cur != -1; cur = parent[cur]) path.push_back(cur);
    reverse(path.begin(), path.end());
    if (path.empty() || path[0] != s) return {};
    return path;
}
```

- Use only when no reachable negative cycle affects the target.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Complexity by representation

Representation	Complexity
Edge list (natural)	$O(nm)$
Adjacency list	still $O(nm)$ overall
Adjacency matrix	$O(n^3)$ if scanning all pairs as edges

Why these costs?

- Bellman–Ford performs up to $n-1$ full relaxation passes over edges.
- With edge list (or list-derived edge iteration), each pass scans m edges $\Rightarrow O(nm)$.
- With matrix-style scanning, one pass may inspect n^2 pairs, so total becomes $O(n^3)$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bellman–Ford

Summary

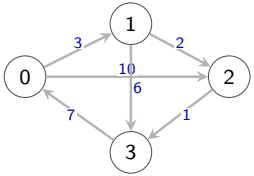
- Use when negative edges may appear.
- Slower than Dijkstra, but more general.
- Extra relaxation pass gives negative-cycle detection.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Definition and example

- Floyd–Warshall computes shortest-path distance for **every ordered pair** (i, j) in a weighted directed graph.
- It also supports path reconstruction (via a `next`/parent-like matrix) when no negative cycle is used in the answer.



Example graph (4 nodes).

$$D = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 10 & 0 & 2 & 3 \\ 8 & 11 & 0 & 1 \\ 7 & 10 & 12 & 0 \end{bmatrix}$$

All pair results (dist, path):

$0 \to 1 : (3,\ 0 \to 1)$
$0 \to 2 : (5,\ 0 \to 1 \to 2)$
$0 \to 3 : (6,\ 0 \to 1 \to 2 \to 3)$
$1 \to 0 : (10,\ 1 \to 2 \to 3 \to 0)$
$1 \to 2 : (2,\ 1 \to 2)$
$1 \to 3 : (3,\ 1 \to 2 \to 3)$
$2 \to 1 : (11,\ 2 \to 3 \to 0 \to 1)$
$2 \to 3 : (1,\ 2 \to 3)$
$3 \to 0 : (7,\ 3 \to 0)$
$3 \to 1 : (10,\ 3 \to 0 \to 1)$
$3 \to 2 : (12,\ 3 \to 0 \to 1 \to 2)$
$2 \to 0 : (8,\ 2 \to 3 \to 0)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Core idea and basic steps

1. Start with weighted adjacency matrix as initial distances.
2. For each intermediate vertex $k = 0..n-1$, update all pairs (i, j).
3. Transition: $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$.

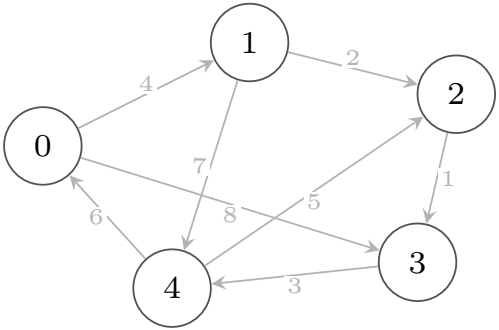
Meaning of the transition

For each pair (i, j), compare: **current best known route** ($dist[i][j]$) vs. **route going through** k ($dist[i][k] + dist[k][j]$). Keep the smaller one.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Problem setup before the matrix walk



Directed weighted graph (5 vertices) for APSP.

- **Goal:** shortest paths for *all pairs* of vertices.
- **Choose representation:** weighted matrix `dist`$[i][j]$.
- **Why matrix?** DP updates every pair (i, j) for each intermediate k.

Before running

- `dist[i][i] = 0`
- missing edge $\Rightarrow \infty$
- no negative cycle for finite shortest-path answers

Shortest paths and MST
Outline
Shortest Path Algorithms
Dijkstra
Bellman–Ford
51 Floyd–Warshall
Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Step 1/5 — initialize matrix and loop variables

A (original) $=$

	0	1	2	3	4
0	0	4	∞	8	∞
1	∞	0	2	∞	7
2	∞	∞	0	1	∞
3	∞	∞	∞	0	3
4	6	∞	5	∞	0

$D^{(init)} =$

	0	1	2	3	4
0	0	4	∞	8	∞
1	∞	0	2	∞	7
2	∞	∞	0	1	∞
3	∞	∞	∞	0	3
4	6	∞	5	∞	0

- **Row** i means source vertex; **column** j means destination vertex.
- `dist[i][j]`: current best-known cost from i to j.
- Diagonal 0: same vertex; ∞: currently no known path.

State variables

- Outer loop: intermediate vertex index `k`.
- Inner loops: pair `(i,j)` currently being updated.
- Rule: `dist[i][j]` $=$ min(`dist[i][j]`, `dist[i][k]` $+$ `dist[k][j]`).

Shortest paths and MST

Outline

Shortest Path Algorithms

Dijkstra

Bellman–Ford

52 Floyd–Warshall

Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Step 2/5 — $k = 0$: first 6 pairs (i, j) from loop start

Loop order (i outer, j inner): $(0,0), (0,1), (0,2), (0,3), (0,4), (1,0), \ldots$

$$A = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & \infty & 8 & \infty \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & \infty & 5 & \infty & 0 \end{array}\right] \qquad D^{(k=0)} = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & \infty & 8 & \infty \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & 10 & 5 & 14 & 0 \end{array}\right]$$

First 6 pairs for $k = 0$

(i, j)	result
$(0,0)$	$\min(0, 0+0) = 0$ (no change)
$(0,1)$	$\min(4, 0+4) = 4$ (no change)
$(0,2)$	$\min(\infty, 0+\infty) = \infty$ (no change)
$(0,3)$	$\min(8, 0+8) = 8$ (no change)
$(0,4)$	$\min(\infty, 0+\infty) = \infty$ (no change)
$(1,0)$	$\min(\infty, \infty+0) = \infty$ (no change)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Step 3/5 — $k = 1$: first 6 pairs (i, j) from loop start

Loop order again: $(0,0), (0,1), (0,2), (0,3), (0,4), (1,0), \ldots$

$$A = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & \infty & 8 & \infty \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & \infty & 5 & \infty & 0 \end{array}\right] \qquad D^{(k=1)} = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & 6 & 8 & 11 \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & 10 & 5 & 14 & 0 \end{array}\right]$$

First 6 pairs for $k = 1$

(i, j)	result
$(0,0)$	$\min(0, 4+\infty) = 0$ (no change)
$(0,1)$	$\min(4, 4+0) = 4$ (no change)
$(0,2)$	$\min(\infty, 4+2) = 6$ (update)
$(0,3)$	$\min(8, 4+\infty) = 8$ (no change)
$(0,4)$	$\min(\infty, 4+7) = 11$ (update)
$(1,0)$	$\min(\infty, 0+\infty) = \infty$ (no change)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Step 4/5 — continue similarly for $k = 2, 3$

Same full scan pattern for each k :

$$\forall i, j,\ \texttt{dist[i][j]} = \min(\texttt{dist[i][j]}, \texttt{dist[i][k]} + \texttt{dist[k][j]}).$$

$$A = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & \infty & 8 & \infty \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & \infty & 5 & \infty & 0 \end{array}\right] \qquad D^{(k=2)} = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & 6 & 7 & 11 \\ 1 & \infty & 0 & 2 & 3 & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & 10 & 5 & 6 & 0 \end{array}\right]$$

- Example updates when $k = 2$: $(1, 3) : \infty \to 3$, $(0, 3) : 8 \to 7$, $(4, 3) : 14 \to 6$.
- Then for $k = 3$, more entries to node 4 improve.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Step 5/5 — final $k = 4$ and last pair

Last pair in the whole run $(k, i, j) = (4, 4, 4)$: `dist[4][4]` $= \min(0, 0{+}0) = 0$ (no change)

$$A = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & \infty & 8 & \infty \\ 1 & \infty & 0 & 2 & \infty & 7 \\ 2 & \infty & \infty & 0 & 1 & \infty \\ 3 & \infty & \infty & \infty & 0 & 3 \\ 4 & 6 & \infty & 5 & \infty & 0 \end{array}\right] \qquad D^{(final)} = \left[\begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 0 & 0 & 4 & 6 & 7 & 10 \\ 1 & 12 & 0 & 2 & 3 & 6 \\ 2 & 10 & 14 & 0 & 1 & 4 \\ 3 & 9 & 13 & 8 & 0 & 3 \\ 4 & 6 & 10 & 5 & 6 & 0 \end{array}\right]$$

How to read the result

- Entry `dist[i][j]`: shortest-path length from i to j.
- Diagonal check: if any `dist[i][i]` < 0, a negative cycle exists.

Shortest paths and MST
Outline
Shortest Path Algorithms
Dijkstra
Bellman–Ford
56 Floyd–Warshall
Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Function signature and return type

```
vector<vector<int>> floydWarshall(vector<vector<int>> dist);
```

- Input `dist`: weighted adjacency matrix (`dist[i][j]` = edge cost, or `INF` if no edge).
- Return: final all-pairs shortest-path matrix.
- In-place update style: we copy/update `dist` through the triple loop.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Important local variables (with example)

```
int n = dist.size(); // number of vertices
const int INF = 1e9; // no-path sentinel
for (int k = 0; k < n; ++k) // allowed intermediate vertices: 0..k
  for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j) { /* relax (i,j) via k */ }
```

- Example init (3 vertices): `dist = [[0,5,INF],[INF,0,2],[4,INF,0]]`.
- At `k=1`, update `dist[0][2]` by checking `min(INF, dist[0][1]+dist[1][2]) = 7`.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Implementation

```
vector<vector<int>> floydWarshall(vector<vector<int>> dist) {
    int n = dist.size(); const int INF = 1e9;
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            if (dist[i][k] < INF)
                for (int j = 0; j < n; ++j)
                    if (dist[k][j] < INF)
                        dist[i][j] = min(dist[i][j], dist[i][k] +
                            dist[k][j]);
    return dist;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Complexity by representation

Representation	Complexity
Adjacency matrix (natural)	$O(n^3)$ time, $O(n^2)$ memory
Adjacency list	typically convert to matrix first
Edge list	typically convert to matrix first

ltsach@hcmut.edu.vn
Thanh-Sach LE

Floyd–Warshall

Summary

- Best for all-pairs queries on small/medium dense graphs.
- Very simple triple-loop implementation.
- Matrix-centric algorithm by design.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Minimum spanning tree

ltsach@hcmut.edu.vn
Thanh-Sach LE

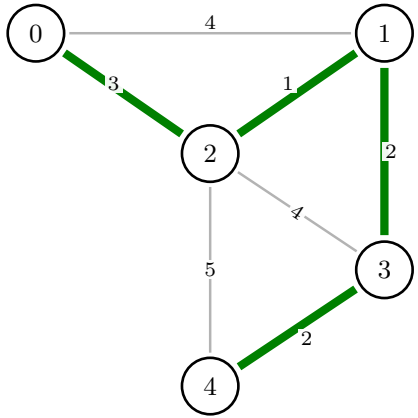
What Is a Minimum Spanning Tree?

- Input: connected, undirected, weighted graph.
- Spanning tree uses exactly $n - 1$ edges and connects all vertices.
- MST minimizes total edge weight.
- If graph is disconnected, result is a minimum spanning forest.

ltsach@hcmut.edu.vn
Thanh-Sach LE

MST Example

- All edges (weight on drawing):
  - 0-1(4), 0-2(3), 1-2(1), 1-3(2)
  - 2-3(4), 3-4(2), 2-4(5)
- **One MST** (thick green): 1-2(1), 1-3(2), 3-4(2), 0-2(3).
- Total weight = 8.
- Gray edges are not in this tree.



Legend: MST edge — omitted from this MST

ltsach@hcmut.edu.vn
Thanh-Sach LE

MST Properties

- Tree has exactly $n - 1$ edges.
- No cycles, still connected.
- Cut property: lightest crossing edge is safe.
- Cycle property: heaviest edge in a cycle is not needed.

ltsach@hcmut.edu.vn
Thanh-Sach LE

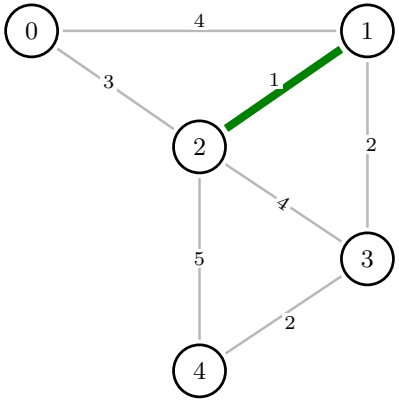
Kruskal Idea

- Sort all edges by weight.
- Add edge if it connects two different components.
- Use DSU (Union-Find) to detect component merge/cycle.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 1/7 — take 1–2



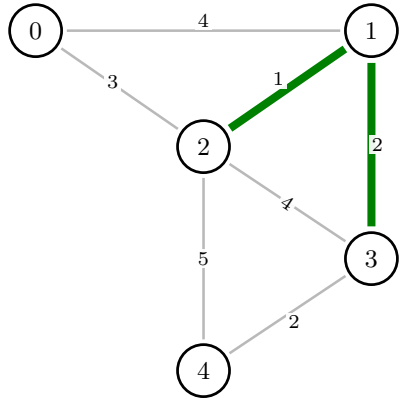
— taken into MST — not yet considered -- skipped (cycle)

Smallest edge first: 1–2 *(1).* `unite(1,2)` *succeeds* ⇒ **take** *(two different components).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 2/7 — take 1–3

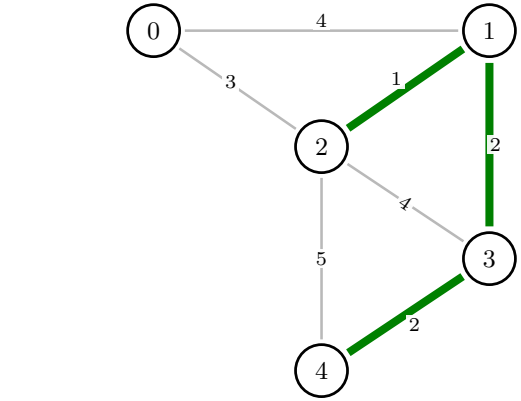


Next: 1–3 *(2). Endpoints in different components* ⇒ **take** *(attach* 3*).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 3/7 — take 3–4

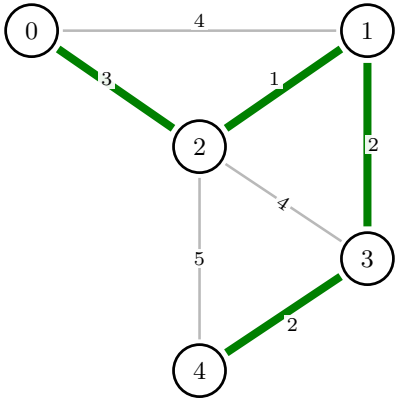


Next: 3–4 *(2).* **Take** — *connects* 4 *without a cycle.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 4/7 — take 0–2



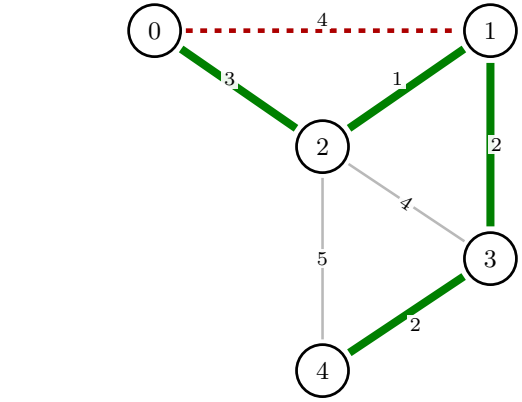
— taken into MST — not yet considered -- skipped (cycle)

Next: 0–2 *(3).* **Take** — *brings* 0 *into the growing tree. Tree now has* $n-1 = 4$ *edges.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 5/7 — skip 0–1

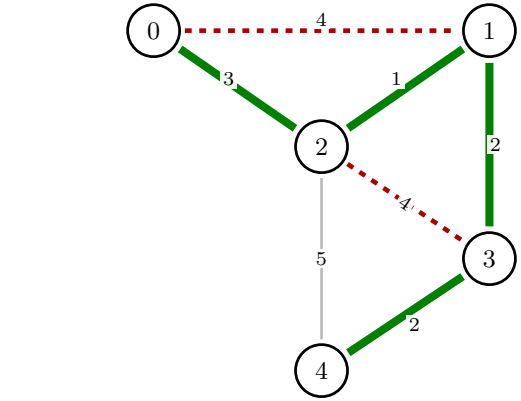


Next: 0–1 *(4).* `find(0)==find(1)` ⇒ **skip** *(would close a cycle).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 6/7 — skip 2–3

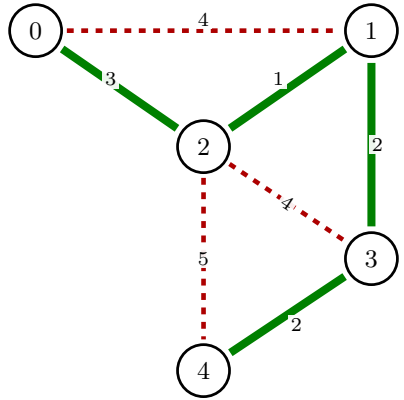


Next: 2–3 *(4). Already connected* ⇒ **skip**.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Step-by-step

Step 7/7 — skip 2–4



— taken into MST — not yet considered -- skipped (cycle)

Next: 2–4 *(5).* **Skip** *again. Heavier edges cannot improve the MST; typically stop once* `used` $= n-1$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DSU: Methods and Meaning

- **DSU** (*Disjoint Set Union*) stores a partition of $\{0, \ldots, n-1\}$ into disjoint **components**. In Kruskal, each component is a connected piece of the forest built so far.
- `DSU(n)`: create n **singleton** sets, one per vertex; each element is its own representative at the start.
- `find(x)`: returns the **representative** (root) of the set that contains x. Two vertices u, v are in the same component iff `find(u) == find(v)`.
- `unite(a,b)`: if a and b lie in **different** components, **merge** those two sets and return `true`. If they are already in the **same** component, return `false` — in Kruskal, that means the current edge would form a **cycle**, so it is skipped.

Shortest paths and MST
Outline
Shortest Path Algorithms
Minimum spanning tree
74 Kruskal
Prim

ltsach@hcmut.edu.vn
Thanh-Sach LE

DSU: Implementation notes

- The next slides show a standard C++ `class DSU` with the same three ideas: `parent` for the forest of roots, `find` with **path compression**, and `unite` with **union by rank** (keeps trees shallow; near–constant time per operation amortized).
- You only need the **semantic** of `find` / `unite` to read Kruskal; the code details support efficiency.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DSU Class

```
class DSU {
private:
    vector<int> parent, rankv;
public:
    DSU(int n) : parent(n), rankv(n, 0) {
        for (int i = 0; i < n; ++i) parent[i] = i;
    }
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

DSU Class (continued)

```
    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rankv[a] < rankv[b]) swap(a, b);
        parent[b] = a;
        if (rankv[a] == rankv[b]) rankv[a]++;
        return true;
    }
};
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Code

```
struct Edge { int u, v, w; };

int kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(),
         [](const Edge& a, const Edge& b) { return a.w < b.w; });

    DSU dsu(n);
    int total = 0, used = 0;
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal Code (continued)

```
    for (const Edge& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            used++;
            if (used == n - 1) break;
        }
    }
    if (used != n - 1) return -1; // disconnected graph
    return total;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

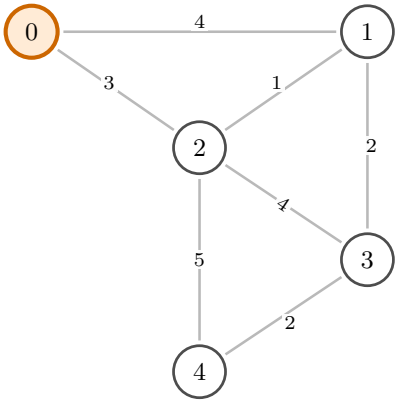
Prim Idea

- Start from any vertex and grow one tree.
- Repeatedly add minimum edge from tree to outside.
- Priority queue keeps candidate crossing edges.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim Step-by-step

Step 1/5 — seed $s = 0$



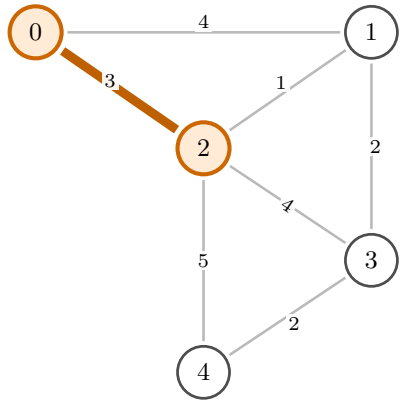
ring = in tree — tree edge — edge added this step — not chosen

Start with tree $T = \{0\}$. Every other vertex is outside; crossing edges from 0 are $0-1$(4) and $0-2$(3) — next step adds the cheaper attach.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim Step-by-step

Step 2/5 — add 0–2



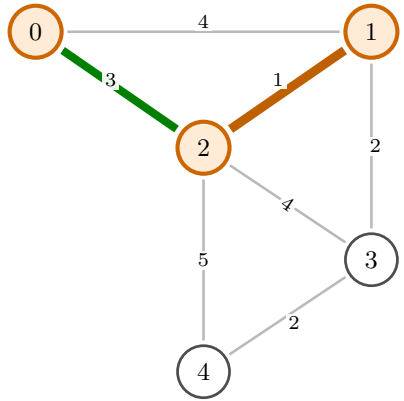
ring = in tree — tree edge — edge added this step — not chosen

Minimum edge leaving $\{0\}$ *is* $0-2$ *(3).* **Add** *it; now* $T = \{0, 2\}$ *and the growing tree has one edge.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim Step-by-step

Step 3/5 — add 1–2



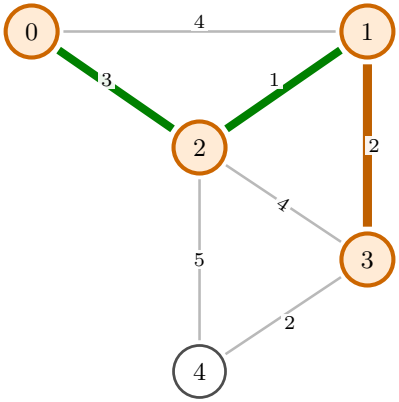
ring = in tree — tree edge — edge added this step — not chosen

Cheapest edge from T to $V \setminus T$: endpoints in $\{0,2\}$ vs. $\{1,3,4\}$ — pick $1-2$ (1).
Add; $T = \{0,1,2\}$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim Step-by-step

Step 4/5 — add 1–3



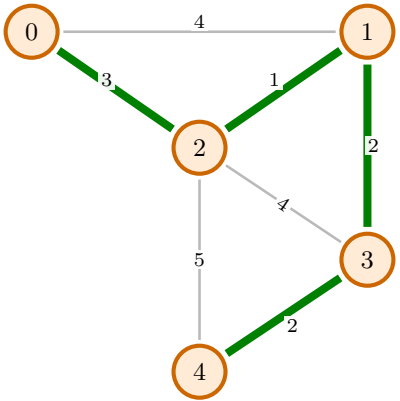
ring = in tree — tree edge — edge added this step — not chosen

Next lightest crossing edge is 1−3 *(2).* **Add***; attach* 3*. Three tree edges, still acyclic.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim Step-by-step

Step 5/5 — add 3–4, done



ring = in tree — tree edge — edge added this step — not chosen

Connect the last vertex with $3-4$ *(2). Tree has* $n-1=4$ *edges; total weight* $3+1+2+2=8$ *(same MST cost as Kruskal on this graph).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue

Function signature

- **Declaration:** `int prim(int n, const vector<vector<pair<int,int>>>& adj);`
- **Return value:** sum of weights of MST edges if the graph is **connected**; `-1` if fewer than n vertices are reachable from the start (disconnected).
- `n`: number of vertices, labeled $0..n-1$.
- `adj`: weighted adjacency list. For an **undirected** graph, each edge (u, v, w) should appear in both `adj[u]` and `adj[v]` as `{v,w}` / `{u,w}` so every neighbor is discoverable from either endpoint.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue

Local state (1)

- `vector<bool> inMST(n, false)` — `inMST[v] == true` iff vertex v is already in the growing tree.
- `int total` — sum of weights of tree edges actually taken (each time we **settle** a vertex, we add the key `w` that popped with it).
- `int count` — how many vertices have been added to the tree; at the end, `count == n` for a connected graph (else `-1`).
- `pq.push({0, 0})` — start from vertex 0 with cost 0 (no edge paid for the seed); another start s would use `pq.push({0, s})`.

Shortest paths and MST

Outline

Shortest Path Algorithms

Minimum spanning tree

Kruskal

87 Prim

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue

Local state (2): min-priority queue

- `priority_queue< pair<int,int>, vector<...>, greater<...> >` — min-heap on `pair`: smallest `.first` first (cheapest frontier attachment).
- Each entry `{w, v}` means: "we can attach v to the tree with an edge of weight at least `w`" (often the best known candidate from some frontier vertex).
- Multiple copies for the same v may exist; when v is already in the tree, `if (inMST[u]) continue;` discards stale pops (same lazy idea as Dijkstra).
- After settling u, the loop `for (auto [v, weight] : adj[u])` relaxes all neighbors not yet in the tree by pushing `{weight, v}`.

Shortest paths and MST
Outline
Shortest Path Algorithms
Minimum spanning tree
Kruskal
88 Prim

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue

Main loop (idea)

- Repeatedly pop the cheapest frontier offer `(w,u)`. If u is new, pay `w`, mark `inMST[u]`, add all outgoing edges from u that lead outside the tree into `pq`.
- **Invariant:** the popped `w` for the first visit to u is the weight of the minimum edge connecting u to the current tree (cut property).
- Stop when `pq` is empty; if `count < n`, the graph was disconnected from the start vertex.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue

```
int prim(int n, const vector<vector<pair<int,int>>>& adj) {
    vector<bool> inMST(n, false);
    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;
    pq.push({0, 0});
    int total = 0, count = 0;
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Prim with Priority Queue (continued)

```
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true;
        total += w;
        count++;

        for (auto [v, weight] : adj[u]) {
            if (!inMST[v]) pq.push({weight, v});
        }
    }
    if (count != n) return -1;
    return total;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

MST Complexity

- Kruskal: sorting dominates $\Rightarrow O(m \log m)$.
- Prim + priority queue + adjacency list: $O(m \log n)$.
- Prim + adjacency matrix: $O(n^2)$ (good for dense graphs).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kruskal vs Prim

Kruskal	Prim
edge-list driven	grow from source component
sort all edges	repeatedly pick best frontier edge
uses DSU	uses priority queue
natural for forest	starts from one component (repeat if needed)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Common MST Mistakes

- Applying MST directly to directed graph.
- Forgetting connectivity requirement for single MST.
- Confusing shortest-path tree with MST.
- Not skipping cycle-forming edges in Kruskal.
- Not checking disconnected graph in Prim/Kruskal.

ltsach@hcmut.edu.vn
Thanh-Sach LE